TALENT ACQUISITION MANAGEMENT SYSTEM

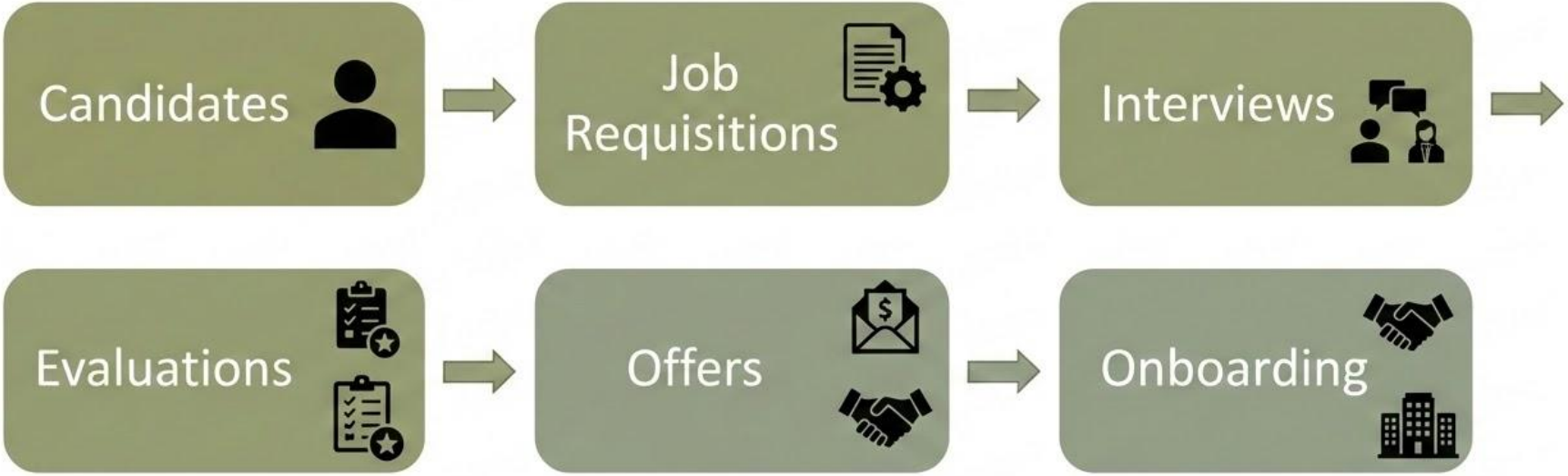
DATABASE DESIGN & IMPLEMENTATION

DADS6700 Data Management for Analytics

Nithyaa Kesavamurthy

Problem Background

Database design for the talent acquisition management system which manages recruitment workflows capturing, managing and tracking:



Summary

Robust Conceptual Design

Fully normalized 3NF data model with supporting EER & UML diagrams.

Dual Technology Implementation

Deployed in both MySQL (Relational) and MongoDB (NoSQL) for flexibility.

Advanced Analytics

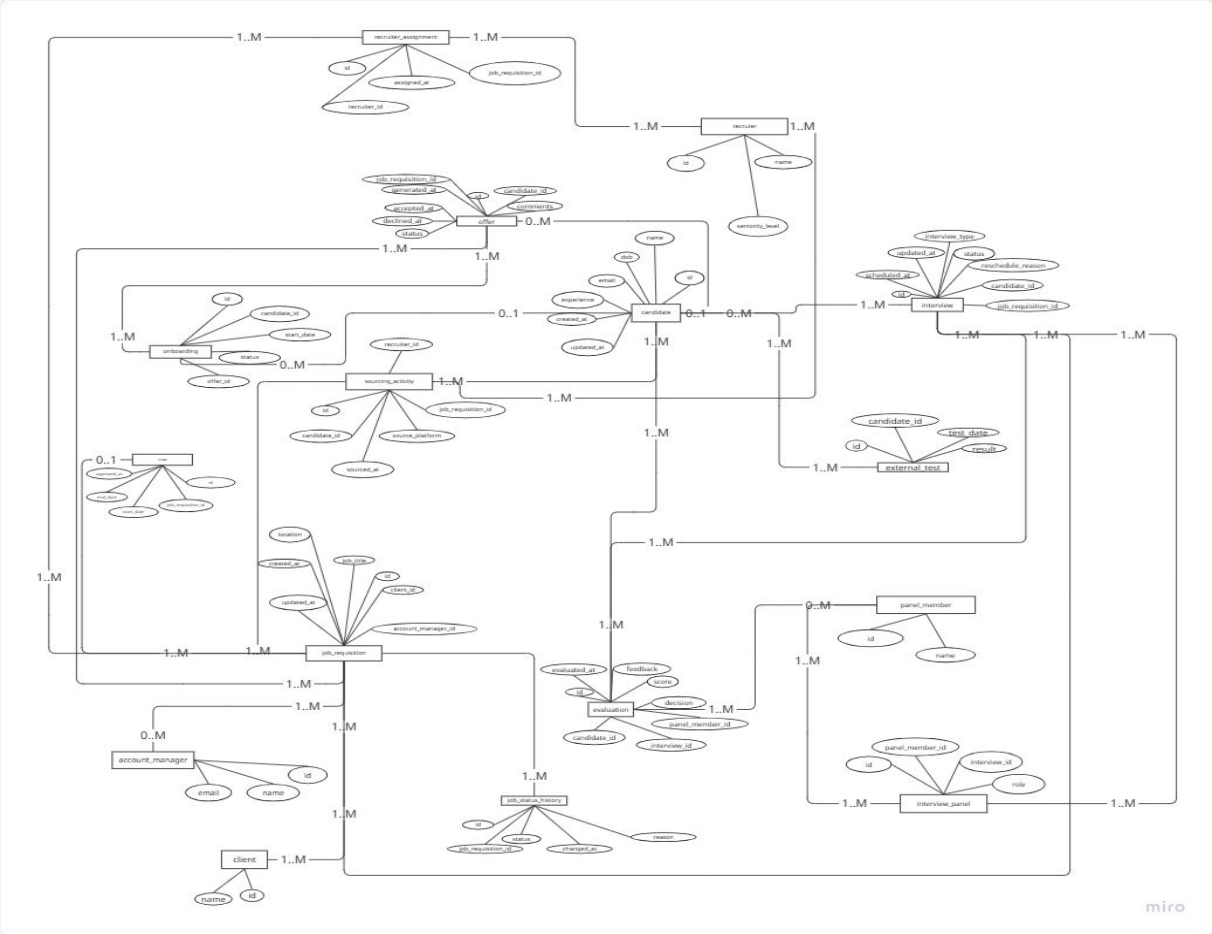
Executed complex SQL & MongoDB queries (CTEs, aggregations) for business insights.

Python Analytical App

Visualized key metrics and insights to support decisions.

Business Impact

Provides a scalable, efficient, and insight-driven recruitment optimization system.

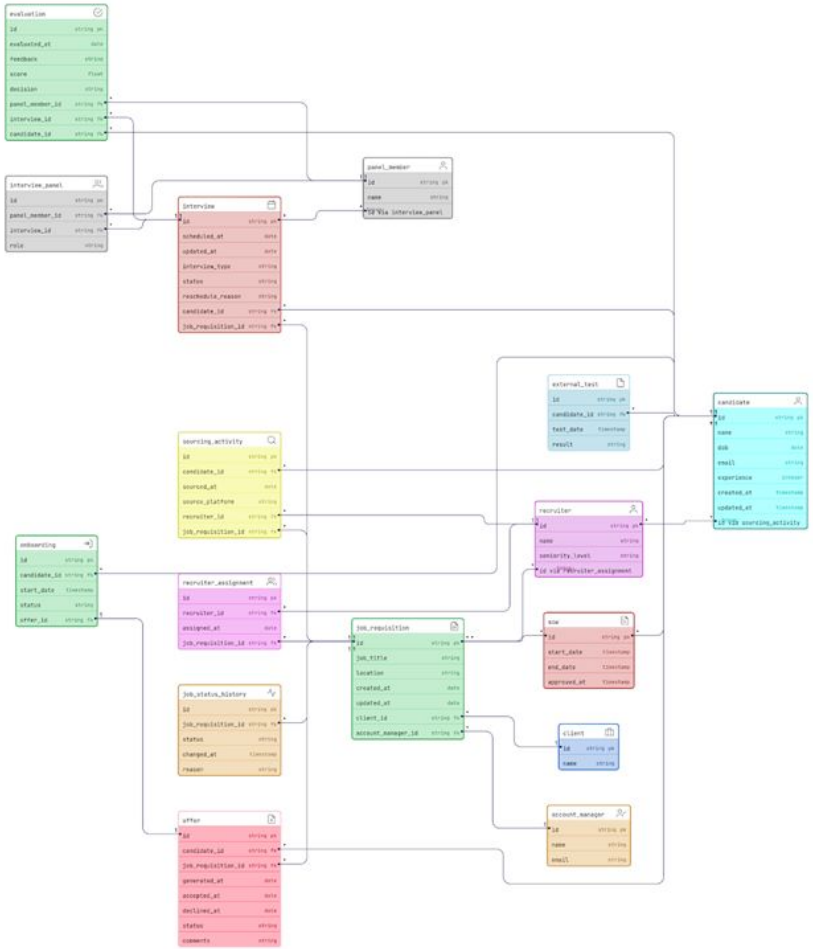


Conceptual model ERDiagram

ER Model – Entities and Relationships

Core Entities	Supporting tables
Candidates	External Test
Job Requisition	Sourcing Activity
Client	Recruiter Assignment
Recruiter	Interview Panel
Interview	Account Manager
Panel Member	SOW
Evaluation	Job Status History
Offer	
Onboarding	

Relationships
Candidate to Job Requisition by Recruiter (1–M via Sourcing Activity)
Client has 1-M Job Requisition
Candidate to Interview (1-M)
Recruiter to Candidate (1-M)
Interview **has** Evaluations by Panel Members (1–M)
Candidate **receives** Offers for Job Requisition (1–M)
Offer **drives** Onboarding activities (1–M)
Interview is **conducted** by Panel Members (M–N via Interview Panel)
Job Requisition is **defined** in SOW and managed by Account Manager (1–M)
Candidate **completes** External Tests (1–M)



UML Class Diagram

Classes: Candidate, Recruiter, Job Requisition, Interview, Evaluation, Offer, Onboarding.

Associations: candidate-interview, job-interview, interview-evaluation.

Relations, Primary Keys, and Foreign Keys

1. Candidate(id PK, name, dob, email, phone, resume_url)
2. Client(id PK, name, industry)
3. Job_Requisition(id PK, job_title, job_description, status, client_id FK→Client.id)
4. Sourcing_Activity(id PK, recruiter_id FK→Recruiter.id, candidate_id FK→Candidate.id, job_requisition_id FK→Job_Requisition.id)
5. Interview(id PK, candidate_id FK→Candidate.id, job_requisition_id FK→Job_Requisition.id, scheduled_at, status)
6. Evaluation(id PK, interview_id FK→Interview.id, panel_member_id FK→Panel_Member.id, score, decision)
7. Offer(id PK, candidate_id FK→Candidate.id, job_requisition_id FK→Job_Requisition.id, status, salary, offered_at)
8. Onboarding(id PK, offer_id FK→Offer.id, activity_name, activity_status)
9. Recruiter(id PK, name, email)
10. Panel_Member(id PK, name, designation)
11. Interview_Panel(id PK, Panel_Member_id FK→ Panel_Member.id, interview_id FK→Interview.id, role)
12. Recruiter_Assignment(id PK, recruiter_id FK→Recruiter.id, assigned_at, job_requisition_id FK→ Job_Requisition.id)
13. sow(id PK, job_requisition_id FK→ Job_Requisition.id, start_date, end_date, approved_at)
14. account_manager(id PK, name, email)
15. Job_Status_History(id PK, job_requisition_id FK→ Job_Requisition.id, status, changed_at, reason)

Normalization

All the tables are fully normalized, conforming to 1NF, 2NF, and 3NF.

All attributes are atomic.

No partial or transitive dependencies.

MySQL Query Validation and Demo Overview

- Implemented the Logical Relational Model in MySQL and validated it through **progressively complex SQL queries**.
- Queries demonstrate **relational integrity**, **advanced analytics**, and **business intelligence extraction**.
- Demo includes simple queries, joins, aggregates, nested, correlated, ALL/ANY, EXISTS, UNION, subqueries.

The SQL Script can be found here.

Sample SQL Queries

Result Grid | Filter Rows: | Export

NAME	SCORE
Leila Verma	4.98
Dallas Chen	4.98

Result Grid | Filter Rows: | Export

NAME	MAX_SCORE
Robin Miller	4.74
Kai Moore	4.49
Micah Nair	4.56
Steve Ross	3.39
Dallas Johnson	3.39
Taylor Harris	4.42
Dallas Chen	4.58
Leila Martinez	4.45
Casey Brown	3.16
Drew Moore	4.7
Vikram Thomas	4.97
Quinn Moore	3.82
Toby Wilson	4.65
Casey Garcia	4.18
Leila Verma	3.68

Query 2 Results: Max Scores vs. Average

≥ ALL (Query 5)

Identifies the **highest-scoring candidate(s)** across the entire database.

Uses `>= ALL (SELECT score FROM evaluation)`.

Insight: Definitively finds top performers without separately computing `MAX()`.

Nested Subquery (Query 2)

Compares each candidate's **maximum score** to the **global average score**.

Uses a nested `SELECT AVG(score)` and `HAVING` filter.

Insight: Quickly flags candidates performing above the overall benchmark.

NoSQL Overview

MongoDB Querying & NoSQL Flexibility

Uses a **document-oriented** model instead of SQL **joins**, leveraging **find()** and **aggregate()** for efficient querying.

Showcases five key query patterns demonstrating filtering, sorting, and aggregation capabilities.

Collections:

- candidate
- job_requisition
- interview
- offer
- recruiter
- client
- account_manager
- evaluation
- external_test
- interview_panel
- onboarding
- panel_member
- recruiter_assignment
- sourcing_activity

NoSQL Queries

```
db.candidate.find(
  { experience: { $gt: 12 } },
  { name: 1, email: 1, experience: 1, _id: 0 )
)
{
  name: 'Micah Kair',
  email: 'shyler.martin13@example.com',
  experience: 14
}
{
  name: 'Jane White',
  email: 'alitor.texra@example.com',
  experience: 14
}
{
  name: 'Gearson Whitow',
  email: 'harper.roe11@example.com',
  experience: 12
}
{
  name: 'Parker Thompson',
  email: 'aatlea.mart1a0c@example.com',
  experience: 15
}
{
  name: 'Vikram Thompson',
  email: 'atco_andercodt@example.com',
  experience: 15
}
{
  name: 'Olipy Navrie',
  email: 'ehcha.khamazht0@example.com',
  experience: 13
}
{
  name: 'Vikram Thomas',
  email: 'atco.smithk1@example.com',
  experience: 13
```

Simple Filter + Projection

- *Action*: Running Query 1
- *Explanation*: Filters candidates with **experience > 12** and projects only **name**, **email**, **experience**, excluding **_id**.

Sorting Query

- *Action*: Running Query 2
- *Explanation*: Retrieves all job requisitions and sorts them by **updated_at (descending)** to surface the most recently modified postings.

Condition-Based Retrieval (Date Filter)

- *Action*: Running Query 3
- *Explanation*: Filters the interview collection for interviews scheduled **on or after Sept 1, 2025** using **$gte**—useful for forecasting workloads.

Key MongoDB Queries (5 Examples)

```
db.offer.aggregate([
  {
    $group: {
      _id: "$status",
      count: { $sum: 1 }
    }
  },
  { $sort: { count: -1 } }
]);
{
  _id: 'Withdrawn',
  count: 13
}
{
  _id: 'Accepted',
  count: 8
}
{
  _id: 'Declined',
  count: 6
}
{
  _id: 'Pending',
  count: 3
}
recruitment_db>
```

```
db.interview.aggregate([
  {
    $group: {
      _id: null,
      earliest_date: { $min: "$scheduled_at"
      latest_date:   { $max: "$scheduled_at"
    }
  }
]);
{
  _id: null,
  earliest_date: 2022-01-11T08:00:00.000Z,
  latest_date: 3025-12-23T08:00:00.000Z
}
```

```
cruiter.find({
: { $regex: "Singh", $options: "
: ObjectId('69331beaac636a11fb5
UUID('57c15145-8f17-481a-86be-fe
: 'Quinn Singh',
iority_level: 'Junior'
ment_db>
```

Aggregation: Group & Count

Action: Running Query 4

Explanation: Groups offer documents by **status** and counts occurrences, then sorts to identify the **most common offer outcome**.

Aggregation: Min & Max (Timeline Summary)

Action: Running Query 5

Explanation: Computes the **earliest and latest interview dates** using $min and $max across all interviews—helpful for timeline analysis.

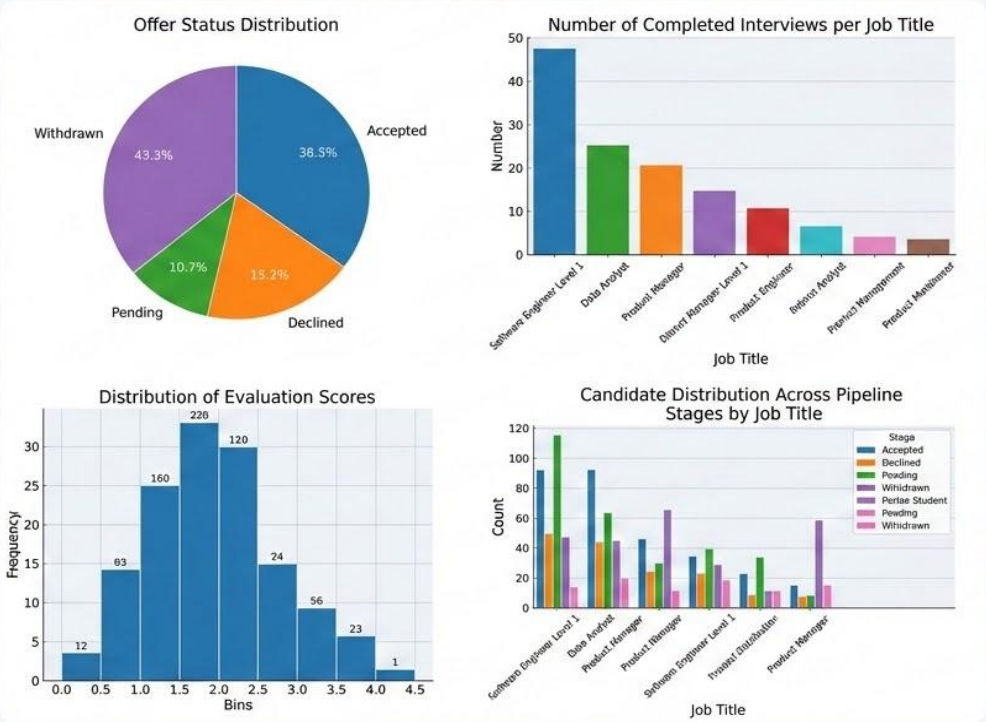
Python App

Python Integration & Analytical Validation

- **Programmatic Access**: Verified that the MySQL database can be seamlessly consumed using Python for analysis and visualization.

- **Data Retrieval Workflow**: Connected via pymysql, executed SQL queries with pandas.read_sql(), and loaded results directly into DataFrames for instant analysis.

Python Visualizations



- **Key Visualizations Generated (6 Total)**
  - **Completed Interviews per Job Title**
    - **Source:** MySQL Query 1 (Join + Aggregate).
    - **Insight:** Identifies funnel efficiency; "Software Engineer Level 1" shows the highest interview completions.
  - **Candidate Distribution Across Pipeline Stages**
    - **Source:** MySQL Query 4 (CASE WHEN + Join)
    - **Insight:** Highlights stage-wise candidate volumes to detect bottlenecks.
  - **High-Scoring Candidates (Recent Comparison)**
    - **Source:** MySQL Query 6 (> ANY)
    - **Insight:** Flags candidates scoring better than any evaluated in the past week.
  - **Average Recruiter Score Analysis**
    - **Source:** MySQL Query 9 (CTE)
    - **Insight:** Shows recruiter effectiveness; top performers include Harper Garcia & Parker Iyer.
  - **Evaluation Score Distribution**
    - **Source:** Simple SELECT
    - **Insight:** Histogram reveals scoring patterns around 2.0 and 3.0–3.5.
  - **Offer Status Distribution**
    - **Source:** Aggregate SELECT
    - **Insight:** Offers a critical metric—43% withdrawn, indicating a major drop-off point.

Summary

Robust Conceptual Design: Fully normalized 3NF data model with supporting EER & UML diagrams.

Dual Technology Implementation: Deployed in both MySQL (Relational) and MongoDB (NoSQL) for flexibility.

Advanced Analytics: Executed complex SQL & MongoDB queries (CTEs, aggregations) for business insights.

Python Analytical App: Visualized key metrics and insights to support decisions.

Business Impact: Provides a scalable, efficient, and insight-driven recruitment optimization system.